Desktop Liveness: Derby Database Task

Oct 21, 2026 · @prateeek

Goal

Make Desktop face liveness store its audit records and read its config through the Desktop Registration Client's existing Apache Derby database, the same way Android already does with SQLite.

- **Do:** use the reg client's existing audit service and the synced global params table.
- **Do not:** create a new database, add new Derby tables, or write liveness data to your own files.
- **Why:** MOSIP judges will check that liveness follows the client's own audit and config framework, which the PS lists under integration ("Existing Registration Client configuration framework").

Class and method names for the Desktop client below are from MOSIP's `registration` repo as best known. Confirm each one in your version of the code before using it.

What liveness stores and where

Only two things touch the database: audit records and the liveness config. Copy the Android design onto the Desktop equivalents.

Data	Android today (reference)	Desktop target (Derby)
Audit records: session start/end, passive decision, challenge result, config applied/rejected, state changes, timings	`AuditManagerLivenessLogger` calls `AuditManagerService.auditWithArguments(...)`; rows land in the client's SQLite audit table	A `LivenessAuditLogger` implementation that calls the Desktop `AuditManagerService` (registration-services); rows land in the Derby audit table
Audit event IDs	New `AuditEvent` entries `REG-LIV-001` to `REG-LIV-006` in clientmanager's `AuditEvent.java`	The same six entries added to the Desktop `AuditEvent` enum (`io.mosip.registration.constants`)
Liveness config	Read from global param `mosip.registration.face.liveness.config` via `GlobalParamRepository`, parsed by `LivenessConfigParser`, last good copy cached, bundled default as fallback	Read the same global param from the Desktop app's global params (synced into Derby, e.g. via `ApplicationContext` or `GlobalParamService`), parse with the same `LivenessConfigParser`, bundled default JSON as fallback

The six audit events to copy exactly:

ID	Name	Type	Used for
REG-LIV-001	FACE_LIVENESS_SESSION	USER_EVENT	Session started, ended, performance timings
REG-LIV-002	FACE_LIVENESS_STATE	SYSTEM_EVENT	State transitions
REG-LIV-003	FACE_LIVENESS_DECISION	SYSTEM_EVENT	Passive decision with mean/min/max score and thresholds
REG-LIV-004	FACE_LIVENESS_CHALLENGE	USER_EVENT	Challenge result
REG-LIV-005	FACE_LIVENESS_CONFIG	SYSTEM_EVENT	Config applied or rejected
REG-LIV-006	FACE_LIVENESS_GATE_BLOCKED	SYSTEM_EVENT	Face capture blocked because liveness had not passed

Tasks

Work through these in order; each one is small, and the first tells you how much is already done.

A. Check what exists

- [ ] Search the Desktop liveness code for anything that logs or stores liveness data outside the audit service: `System.out`, `printStackTrace`, file writes, a separate DB or JSON file. List what you find.
- [ ] Search for hardcoded thresholds, challenge counts, timeouts or retry limits. These must come from config instead.

B. Audit records into Derby

- [ ] Add the six `REG-LIV-00x` entries to the Desktop `AuditEvent` enum, with the same IDs, names and types as Android.

- [ ] Write a Desktop `LivenessAuditLogger` that maps each `LivenessAuditRecord` type to its event (same mapping as Android's `AuditManagerLivenessLogger.eventFor`) and calls the Desktop `AuditManagerService`.
- [ ] Pass `record.toCompactString()` as the description, so the text matches Android.
- [ ] Wrap the audit call in try/catch: a failed audit write must never break face capture. Log only the exception class name.
- [ ] Plug this logger into the Desktop liveness controller so every session, decision, challenge and config event is written.
- [ ] Write a `FACE_LIVENESS_GATE_BLOCKED` audit when face capture is refused because no valid liveness pass exists.

C. Config from the Derby global params

- [ ] Read global param `mosip.registration.face.liveness.config` through the Desktop client's existing global param access. Do not query Derby directly.
- [ ] Parse and validate it with the shared `LivenessConfigParser` (including the SHA-256 `integrity` check from `CanonicalJson`).
- [ ] If it is missing or invalid: keep the previous valid config, or fall back to the bundled `default_liveness_config.json`, and write a `FACE_LIVENESS_CONFIG` audit saying it was rejected.
- [ ] Write a `FACE_LIVENESS_CONFIG` audit with `configVersion` whenever a config is applied.
- [ ] Use the policy per workflow (resident, operator, supervisor) for thresholds, challenge types, count, timeout and retries.

Rules

These match the PS privacy requirements and what the Android side already enforces.

- **No face images anywhere in the database or logs.** No frames, crops, thumbnails or embeddings. The SBI/MDS capture stays the only stored biometric.
- **Scores only in audit records.** Scores, thresholds, model names and attack types may appear in audit descriptions, never on screen.
- **No new tables, no schema change.** Adding Derby tables needs upgrade SQL scripts and is easy to get wrong. If something seems to need a table, ask first.
- **No direct SQL.** Go through the client's existing services and DAOs.
- **Fail closed.** If config or audit setup fails, liveness must not silently pass.

How to test and when it is done

The task is done when every check below passes and you have sent the screenshots back to the team.

- [ ] **Unit test the logger:** with a mocked `AuditManagerService`, each `LivenessAuditRecord` type produces the right `REG-LIV-00x` event, and a throwing audit service does not break the session.
- [ ] **Unit test the config:** a valid config is applied; a config with a bad integrity digest, an unknown key or an out-of-range value is rejected and the previous one stays active.
- [ ] **Audit rows appear:** run one resident capture, one operator login and one supervisor approval, then confirm the `REG-LIV` audit entries were written (through the client's audit view, logs or a debug query via the existing DAO).
- [ ] **Config change works:** change `passiveThreshold` or `challengeCount` in the global param, sync, restart, and confirm the new value is used and a `FACE_LIVENESS_CONFIG` audit was written.
- [ ] **Offline works:** sync once, disconnect the network, restart the client and run a capture. Liveness must still use the last synced config.
- [ ] **No images stored:** search the audit entries and log files for image data or base64 strings; there must be none.
- [ ] Push the code and send the team: the list from task A, screenshots of the audit entries, and anything that did not work.